

Plan PRO — RAG aplicado a Mageam

Versión: 1.0 **Fecha:** 2026-04-25 **Autor:** David Cabezas **Estado:** Draft — pendiente sign-off **Alcance:** Indexar histórico operativo (OTs cerradas, RCAs, WRs validados, captures, comentarios de cierre) y wirear a 3 superficies UX: Equipment Chat, Create WR (AI Assist), Scheduling.

1. Objetivo de negocio

Convertir el "AI-Powered Platform" del logo en valor operativo medible:

Actor	Hoy	Con RAG
Técnico (Field Capture)	Sugerencias genéricas tipo "inspect, stop, contact"	"Última vez OT-2025-1102: bloqueo + drenaje + cambio sello 770-225, 4h"
Planner (Scheduling)	Asigna técnicos a ojo, estimación HH heurística	"Pérez cerró 3/3 similares en 6h media, libre martes. p90 14h"
Manager (Equipment Chat)	Pregunta sólo si conoce el TAG exacto	Pregunta en lenguaje natural, respuesta cita OT/RCA/manual

Diferenciador comercial Goldfields: SAP PM no tiene esto integrado al flujo. Justifica precio premium vs CMMS commodity.

2. Stack técnico (decisión)

Pieza	Elegido	Por qué
Embeddings	<code>sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2</code>	Local, gratis, multilingüe (ES+EN), 384-dim, ~50ms/query CPU. Sin API keys.
Vector DB	LanceDB	1 archivo en <code>data/</code> , soporta filtros por metadata, sin server. Compatible con volume Docker existente.
Reranker (Fase 5)	<code>cross-encoder/ms-marco-MiniLM-L-12-v2</code>	+30% precisión sobre top-K, también local
LLM generador	Claude (<code>ai_provider_service.py</code> ya existe)	Sin cambio
Hybrid search (Fase 5)	rapidfuzz + BM25Okapi	Mata casos de TAG/código exacto que vectorial pierde

Rechazado: - OpenAI embeddings → API keys, costo recurrente, datos de cliente salen del VPS - Pinecone/Weaviate → infra adicional para algo que cabe en 1 archivo - Llama local → consume RAM que el VPS no tiene libre

3. Datos a indexar

Fuente	Tabla	Campos	Volumen estimado	Prioridad
OTs cerradas	<code>work_orders</code>	description, problem_description, completion_notes, operations[].description	~5K/año/planta	P0
RCAs	<code>rca_analyses</code>	problem_statement, root_causes[], actions[], conclusions	~50/año	P0
WRs validados	<code>work_requests</code>	description, ai_classification.summary, circumstances	~10K/año	P0
Field captures	<code>field_captures</code>	comments, voice_transcript, photo_ocr	~30K/año	P1
FMECA failure modes	<code>failure_modes</code>	mode_description, effect, causes, controls	~500 estáticos	P1
Manuales PDF (futuro)	<code>equipment_manuals</code> (no existe aún)	chunks de texto OCR	TBD	P2

Metadata por chunk (para filtros en retrieval):

```
{
  "chunk_id": "ot_50123_op_2",
  "source_type": "work_order_op",    # work_order, rca, wr, capture, fmeca, manual
  "source_id": "OT-2026-50123",
  "equipment_tag": "BRY-SAG-ML-002",
  "plant_id": "OCP-JFC1",
  "date": "2025-11-14",
  "priority": "P2",
  "status": "CLOSED",
  "text": "...",
  "vector": [0.12, -0.34, ...]    # 384-dim
}
```

Chunk format (texto que se embebe):

[OT-2026-50123 | BRY-SAG-ML-002 | P2 | 2025-11-14]

Falla: Fuga de aceite trunnion bearing, presión hidráulica baja.

Causa raíz: Sello 770-225 fatigado por sobrecalentamiento.

Acción: Bloqueo, drenaje, cambio sello, llenado ISO VG 220.

HH: 4.5 | Técnico: Mendoza | Repuestos: 770-225 (1), aceite 20L

Encabezado estructurado mejora retrieval (Anthropic contextual retrieval pattern).

4. Arquitectura



Punto clave: el RAG **NO** reemplaza `_gather_equipment_context`. Lo complementa. Datos exactos del tag (FMEA, criticidad, backlog) siguen viniendo del DB dump. Texto libre (descripciones, comentarios, RCAs) viene del RAG.

5. Fases (entregables y validación)

Fase 0 — Setup (0.5 día)

Entregables: - `requirements.txt` += `sentence-transformers==3.3.1` , `lancedb==0.18.0` , `rapidfuzz==3.10.1` - `data/rag_index.lance/` agregado a volume Docker `ocp_db_data` - Modelo MiniLM pre-bajado en build del Dockerfile (evita descarga en runtime) - Feature flag env var `RAG_ENABLED=true|false` (default false)

Validación: container builds, app levanta con flag off (no regression).

Riesgo: modelo MiniLM ~120MB sube el image size. Mitigación: layer cache, no excluir en `.dockerignore`.

Fase 1 — Indexer (1 día)

Entregables: - `api/services/rag_indexer.py` con funciones: - `build_full_index()` — corre 1 vez, indexa todo el histórico - `incremental_update(since: datetime)` — para cron nightly - `delete_source(source_id)` — para cuando una OT se cancela - Comando CLI `python -m api.services.rag_indexer --full` (admin/manager) - Endpoint `POST /api/v1/admin/rag/reindex` (admin only) con BackgroundTasks - Cron en `api/main.py` (APScheduler ya está en stack) que corre `incremental_update` 4am

Validación: - Sobre seed data: index inicial debe tener N chunks > 0 - `incremental_update` corrida 2 veces: idempotente (no duplica) - Test pytest `tests/test_rag_indexer.py` : 5 OTs mockeadas → 5+ chunks indexados

Riesgo: primer build full sobre 5K OTs puede tardar ~5 min. Aceptable porque corre 1 sola vez.

Fase 2 — Retrieval service + endpoint raw (0.5 día)

Entregables: - `api/services/rag_service.py` : - `retrieve(question, filters={}, k=8) -> List[Chunk]` - `format_chunks_for_prompt(chunks) -> str` — devuelve texto numerado con citas - `POST /api/v1/ai/rag-search` (admin/manager/engineer): `json Request: { "question": "...", "equipment_tag": "...", "k": 8 }` `Response: { "chunks": [{ "text": "...", "source_id": "OT-X", "score": 0.87 }], "elapsed_ms": 142 }` - Página `/admin/rag-debug` para que admin pruebe queries y vea retrieval

Validación: - 10 queries golden hechas a mano (David + Jorge): top-3 chunks deben ser "obviamente relevantes" - Latencia p50 < 200ms, p99 < 500ms en VPS

Fase 3 — Wire al Equipment Chat (0.5 día)

Entregables: - `equipment_chat_service.py` : agregar `_retrieve_similar_cases(question, equipment_tag)` que llama al RAG con filtro `equipment_tag` . Si flag off, retorna `[]` . - Prompt actualizado: `Equipo actual (datos exactos): {context_dump} Casos históricos similares: {rag_chunks}`
`Pregunta: {q} Responde citando OT-XXX cuando uses casos históricos.` - Frontend
`EquipmentChat.jsx` : parser de respuestas que detecta `OT-XXXX` y los renderiza como chips clickeables que navegan a `/work-orders?id=XXX` .

Validación: - Mismas 5 preguntas con flag on/off → respuesta con flag on debe citar al menos 1 OT real - Toggle flag off → comportamiento idéntico al actual (no regression)

Fase 4 — Wire al Create WR / AI Assist (1 día) ★ Mayor valor visible

Entregables: - Nuevo endpoint `POST /api/v1/ai/suggest-wr-fields` : `json Request: { "description": "fuga de aceite trunnion", "equipment_tag": "BRY-SAG-ML-002", "estado_equipo": "Operating" }`
`Response: { "suggested_title": "Reemplazo sello trunnion bearing CV-024", "suggested_actions": "...con citas...", "suggested_priority": "P2", "priority_rationale": "7/8 OTs similares fueron P2", "estimated_hours": { "p50": 4.5, "p90": 11 }, "likely_parts": ["770-225", "ISO VG 220"], "cited_sources": ["OT-2025-1102", "OT-2024-0381"] }` - `FieldCapture.jsx` botón "AI Assist" llama al nuevo endpoint en lugar del genérico actual. - Render con chips citables abajo de "Suggested Actions". - A/B mediante feature flag por usuario (admin puede activar para 1 técnico de prueba).

Validación: - Demo con Jorge: 5 descripciones reales → debe sugerir título sensato + acciones citando OTs. - Métrica adopción: % WRs creadas usando AI Assist vs manual (instrumentar evento `wr.ai_assist.accepted`).

Riesgo: sugerencias malas pueden inducir errores. Mitigación: NUNCA auto-llenar — siempre el usuario ve sugerencia y aprueba/edita.

Fase 5 — Wire al Scheduling (1 día)

Entregables: - Endpoint `POST /api/v1/ai/similar-cases-summary` : - Input: OT id (que aún no está asignada) - Output: `{ recommended_technician, hh_estimate_p50, hh_estimate_p90, parts_likely, similar_ot_ids[] }` - `Scheduling.jsx` / `Planning.jsx` : tooltip / panel lateral en cada OT card con la recomendación. - Botón "Aplicar sugerencia" — llena los campos del scheduler con un click.

Validación: - 10 OTs nuevas → planner las asigna primero a ojo, después con sugerencia → comparar tiempo de decisión y diff.

Fase 6 — Hardening (0.5-1 día, sólo si Fase 4 demuestra valor)

Entregables: - Hybrid search: BM25 sobre `equipment_tag` , `priority` , códigos SAP exactos. Combina top-20 vectorial + top-20 BM25 → RRF (reciprocal rank fusion). - Reranker cross-encoder sobre top-20 → top-5 final. +30% precisión por benchmark. - Eval suite: `tests/test_rag_eval.py` con 30 queries golden +

ground truth, mide recall@5 y citation accuracy. - Métricas Prometheus: `rag_retrieval_latency_ms` , `rag_chunks_returned` , `rag_citation_used` (cuántas veces el LLM usó las citas). - Dashboard `/admin/rag-metrics` con esos KPIs.

6. Esfuerzo total

Fase	Días	Entregable visible
0 — Setup	0.5	Container con flag off
1 — Indexer	1.0	Index lleno + cron
2 — Retrieval raw	0.5	<code>/admin/rag-debug</code>
3 — Equipment Chat	0.5	Chat cita OTs reales
4 — Create WR 	1.0	AI Assist con citas
5 — Scheduling	1.0	Sugerencias en calendar
6 — Hardening	1.0	Métricas + reranker
Total	5.5 días	

Camino crítico para demo Goldfields: Fases 0+1+2+4 = **3 días**. El resto en sprint siguiente.

7. Lo que NO entra en este plan

- Fine-tuning del modelo de embeddings (overkill para volumen actual)
- LLM local — seguimos llamando Claude API, sólo el retrieval es local
- Multi-tenant aislamiento de índices (todos los plant_id en mismo lance file con filtro — suficiente por ahora)
- RAG sobre logs en tiempo real (out of scope, otra arquitectura)
- Indexar manuales PDF — Fase 7 cuando Goldfields suba sus PDFs

8. Riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Calidad sugerencias decepcionante en demo	Media	Alto	Cherry-pick 5 demos con seed data, validar antes; flag off si sale mal
LanceDB corrupción del archivo	Baja	Medio	Backup nightly del <code>data/rag_index.lance</code> junto a SQLite
MiniLM no entiende jerga técnica chilena	Media	Medio	Fase 6 reranker mejora; long-term fine-tune con data del cliente
VPS RAM insuficiente con modelo cargado	Baja	Alto	MiniLM ~150MB RAM. Backend ya usa ~400MB, hay margen. Probar antes.
Privacidad — embeddings filtran info	Baja	Alto	Modelo es local, datos no salen del VPS. Documentar en SECURITY_SCOPE.

9. Acceptance criteria (Fase 4 demo)

Para considerar la inversión justificada:

- [] 80%+ de las sugerencias en 20 queries golden son "aceptables" (juicio David+Jorge)
- [] Latencia end-to-end < 3s desde click "AI Assist" a sugerencia visible
- [] Cero errores de regresión en flujos sin RAG (flag off)
- [] Demo a Goldfields aprobada con feedback "esto sí me sirve"

Si NO se cumplen → flag off en prod, no se ramp. Documentamos lessons learned y se descarta o pivotea.

10. Próximos pasos inmediatos

1. **Sign-off David + (Jorge si quiere)** sobre este plan
2. Crear branch `feature/rag-foundation`
3. Fase 0: PR `chore: add sentence-transformers + lancedb deps`
4. Empezar Fase 1 (indexer) en sprint corriente

11. Referencias

- Anthropic Contextual Retrieval: <https://www.anthropic.com/news/contextual-retrieval>
- LanceDB docs: <https://lancedb.github.io/lancedb/>
- Sentence-Transformers MiniLM multilingual: <https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2>
- Caso de estudio interno: [Ayudas/GUIA_RAG.md](#) (Goldfields PMO GraphRAG)